

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION with Open CV

COURSE CODE: DSA02

COURSE OUTCOME COVERED - CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 20%

Name / Reg No: Tamilselvan sasikumar (192524134)

Case Study Answers

1. Preprocessing and Feature Detection

A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent. Analyze the issue and explain how preprocessing improves feature detection performance.

Analysis of the Problem

Feature detectors such as Harris corners, SIFT, and edge-based methods rely on local intensity gradients to locate distinctive points. When lighting varies between captures, the same physical point can produce very different pixel intensities and gradient magnitudes. This causes three problems: (a) some genuine features fall below the detector's gradient threshold and are missed, (b) shadows and highlights create spurious gradients that are falsely detected as features, and (c) the same scene photographed under different illumination yields a non-repeatable feature set, which directly breaks any later matching or recognition step that assumes the same features appear in both images.

Solution / Explanation

Preprocessing normalizes the intensity distribution before feature detection is attempted:

- Histogram Equalization / CLAHE (Contrast Limited Adaptive Histogram Equalization) redistributes intensity values so contrast is consistent across the image, including in local regions with shadows.
- Gamma Correction compensates for non-linear illumination differences (e.g., under- or over-exposed regions).
- Gaussian Smoothing suppresses high-frequency illumination noise without destroying true structural edges.
- Intensity Normalization scales pixel values to a common range so gradient magnitudes are comparable across images.

Conclusion

These steps stabilize gradient magnitude and direction prior to detection, which increases the repeatability of detected features across images taken under different lighting, reduces false positives from shadows/highlights, and ultimately improves the reliability of any downstream matching or recognition pipeline.

2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching. Analyze why edge detection is insufficient and justify the use of corner detection in this case.

Analysis of the Problem

Edge detectors (e.g., Sobel, Canny) locate pixels where intensity changes sharply in one direction — perpendicular to the edge. Along the edge itself, intensity is nearly constant, so every point on a straight edge looks identical to its neighbors. This is the classical aperture problem: a small window sliding along an edge cannot determine a unique position, because the edge provides only 1-D positional information (across the edge) and no constraint along it. As a result, edge pixels are not distinctive enough to serve as reliable, uniquely identifiable keypoints for matching between two images.

Solution / Explanation

Corner detection (Harris, Shi-Tomasi, FAST) solves this by examining intensity change in two orthogonal directions simultaneously.

- A corner exhibits significant intensity variation in every direction, so its position is constrained in both x and y — giving it a unique, repeatable location.
- The Harris response function uses the eigenvalues of the local gradient structure (second-moment) matrix; large values in both directions indicate a true corner, not just an edge.
- Corners occur naturally at the boundaries of textured regions and at edge junctions, so they are well distributed across an image that has both straight edges and texture.

Conclusion

Because corners are uniquely localizable in two dimensions, they are far better suited as keypoints for matching than points along a straight edge, and are justified as the correct feature type for this image.

3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results. Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

Analysis of the Problem

The Hough Transform detects lines by mapping edge points into a parameter (ρ, θ) accumulator space and searching for peaks where many points vote consistently. Noise corrupts this process in two ways: (a) it produces spurious edge pixels during the edge-detection stage that precedes the Hough vote, and each of these casts an incorrect vote, raising the background noise floor in the accumulator, and (b) it can broaden or split what should be a single sharp peak into several smaller nearby peaks, making it harder to identify the true line and increasing the chance of false or fragmented line detections.

Solution / Explanation

Both the preprocessing stage and the Hough parameters can be tuned to counter this:

- Apply a Gaussian or median filter before edge detection to suppress noise while retaining true edges.
- Use Canny edge detection with proper hysteresis thresholds (rather than a simple gradient threshold) so weak, noise-induced edges are discarded while connected true edges are retained.
- Increase the accumulator vote threshold so only strong, consistent line evidence registers as a detection.
- Coarsen the (ρ, θ) resolution slightly, or apply non-maximum suppression on the accumulator, so nearby noisy peaks merge into a single dominant peak.
- Use the Probabilistic Hough Transform, which only samples a subset of edge points, reducing sensitivity to isolated noisy pixels.

Conclusion

Combining noise-aware preprocessing with careful parameter selection restores sharp, well-separated accumulator peaks and significantly improves line-detection accuracy.

4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively. Analyze why scale invariant methods perform better in this scenario.

Analysis of the Problem

Traditional matching techniques (e.g., fixed-window template matching or basic corner descriptors) assume that corresponding features appear at a similar spatial scale and orientation in both images. When the object is captured at a different scale, a fixed-size window either captures too little structure (image zoomed in) or too much irrelevant background (image zoomed out), so the descriptor computed no longer represents the same physical feature. Similarly, rotation changes the direction of local gradients, so a descriptor built assuming a fixed orientation no longer aligns between the two images. Both effects break the one-to-one correspondence that matching relies on.

Solution / Explanation

Scale-invariant methods such as SIFT and SURF are explicitly designed to counter this:

- A scale-space pyramid (built using Difference-of-Gaussians in SIFT) detects keypoints at multiple scales, so the same physical feature is found at whichever scale it appears in each image.
- Each keypoint is assigned a dominant orientation from the local gradient histogram, and the descriptor is computed relative to this orientation — making it rotation-invariant.
- The descriptor itself (a normalized histogram of gradients) is computed in scale-normalized local coordinates, so its representation is consistent regardless of the image's overall scale.

Conclusion

Because keypoint location, orientation, and descriptor are all computed relative to the feature's own detected scale and dominant direction, scale-invariant methods produce matching descriptors for the same physical feature even when the two images differ in zoom level and rotation.

5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy. Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

Analysis of the Problem

When a large number of features are extracted per image (e.g., high-dimensional descriptors or raw pixel vectors), many of the resulting dimensions are highly correlated with one another and therefore carry redundant information. This inflates storage and computation cost for matching or classification, and can even harm accuracy due to the curse of dimensionality, where distance-based comparisons become less meaningful as dimensionality grows.

Solution / Explanation

Principal Component Analysis addresses this directly:

- PCA computes the covariance matrix of the feature vectors and finds its eigenvectors (principal components), which are the orthogonal directions of maximum variance in the data.
- Features are then projected onto only the top-k principal components (those with the largest eigenvalues), discarding directions that contribute little variance — these low-variance directions are usually noise or redundancy rather than useful signal.
- Because the principal components are uncorrelated by construction, the redundancy between original feature dimensions is eliminated in the reduced representation.

Conclusion

The result is a compact feature vector that retains most of the original discriminative information while requiring far less storage and computation, which directly speeds up subsequent matching, indexing, or classification stages.

6. Preprocessing Impact on Edge Detection

A system performs edge detection on raw images containing noise and uneven illumination, resulting in fragmented edges. Analyze the problem and explain how preprocessing improves edge detection accuracy and continuity.

Analysis of the Problem

Edge detectors mark a pixel as an edge when the local gradient magnitude exceeds a threshold. Noise causes random small gradient fluctuations along a true edge; wherever the noise happens to push the local gradient below threshold, that segment of the edge is missed, breaking a continuous boundary into disconnected fragments. Uneven illumination compounds this: as brightness varies gradually across the scene, the gradient magnitude of the same physical edge also varies, so parts of it may fall below the global threshold in dim regions even though the edge is genuinely there.

Solution / Explanation

Preprocessing restores a stable, uniform gradient response along true edges:

- Gaussian smoothing reduces the random noise fluctuations that break edges into fragments, without significantly blurring genuine structure.
- Illumination correction (histogram equalization, CLAHE, or background/illumination-field subtraction) evens out the brightness so the same edge produces a similar gradient magnitude throughout the image.
- Canny's hysteresis thresholding (a high and a low threshold) helps connect edge fragments — a pixel just below the strong threshold is still kept if it is connected to a strong edge pixel.
- Morphological closing can be applied after edge detection to bridge small remaining gaps between fragments.

Conclusion

With these steps, edges become continuous and well-defined, improving both the accuracy of edge localization and the structural completeness needed for later processing such as Hough-based line/shape detection.

7. Corner Detection vs Edge Detection in Tracking

A tracking system uses edge detection but fails to track moving objects accurately across frames. Analyze why edges are insufficient and justify the use of corner detection for reliable tracking.

Analysis of the Problem

Object tracking across frames requires establishing an unambiguous correspondence for the same physical point from one frame to the next, typically via optical flow. Along a straight edge, the aperture problem means the apparent motion can only be measured perpendicular to the edge — motion along the edge direction is invisible to a local window, so the estimated displacement vector is ambiguous or incorrect. Edges are also less stable under small viewpoint or lighting changes between consecutive frames, further degrading correspondence.

Solution / Explanation

Corner points resolve this ambiguity and are the standard basis for tracking algorithms:

- At a corner, intensity varies strongly in two independent directions, so the local motion is fully constrained (both x and y displacement can be recovered), unlike on an edge.
- The Shi-Tomasi 'Good Features to Track' criterion explicitly selects corners whose structure matrix has two large eigenvalues, guaranteeing the point can be tracked reliably with the Kanade-Lucas-Tomasi (KLT) tracker.
- Corners tend to be more repeatable and stable landmarks (e.g., object boundaries, textured surface junctions) than points on a smooth edge.

Conclusion

Because corners give a fully determined 2-D displacement rather than an ambiguous 1-D constraint, using corner detection instead of relying purely on edges makes frame-to-frame correspondence unambiguous, resulting in accurate and reliable tracking of moving objects.

8. Hough Transform for Shape Detection

A system uses Hough Transform to detect circular shapes but fails in complex backgrounds. Analyze the limitations and explain how preprocessing or parameter tuning can improve detection.

Analysis of the Problem

Circle detection via the Hough Transform requires searching a 3-D parameter space (center coordinates a , b , and radius r), which is inherently more computationally expensive and more sensitive to noise than line detection's 2-D space. In a complex background, texture and clutter generate many unrelated edge points; each casts votes across a range of possible circle parameters, raising the noise floor of the accumulator and producing multiple false peaks. This can cause circles to be missed (true peak buried among false ones) or false circles to be reported (spurious peak crosses the detection threshold).

Solution / Explanation

Improvements target both the input image and the algorithm's parameters:

- Preprocess with noise reduction (Gaussian blur) and use Canny edge detection with well-tuned thresholds so only strong, genuine boundaries contribute edge points.
- Use gradient direction information at each edge point (as in the Hough Gradient Method used by OpenCV's `HoughCircles`) so votes are cast only along the direction consistent with a circle, reducing the search space and false votes.
- Constrain the radius search range (`minRadius`, `maxRadius`) using domain knowledge of the expected object size, which shrinks the parameter space and false-positive rate.
- Increase the accumulator threshold (votes required) and the minimum distance between detected circle centers to suppress weak or duplicate detections.

Conclusion

Together, noise-aware preprocessing and informed parameter tuning substantially reduce false votes from background clutter, allowing the true circular shapes to be detected reliably even in complex scenes.

9. Feature Matching under Illumination Changes

Two images of the same scene are captured under different lighting conditions, causing feature matching errors. Analyze why matching fails and explain how preprocessing or robust descriptors improve performance.

Analysis of the Problem

Simple matching techniques that compare raw pixel intensities (e.g., sum of squared differences or normalized cross-correlation on raw values) assume the same physical point produces similar intensity in both images. Under different illumination, absolute intensities shift significantly — a point that was bright in one image may appear comparatively dark in the other — so intensity-based similarity measures produce poor or incorrect matches even though the underlying structure is identical.

Solution / Explanation

The failure can be addressed both by conditioning the images and by choosing descriptors that are inherently tolerant to lighting change:

- Preprocessing with histogram equalization or intensity normalization brings both images to a comparable brightness/contrast distribution before matching.

- Gradient-based, normalized descriptors such as SIFT and SURF compute histograms of local gradient orientation rather than raw intensity, and normalize the descriptor vector — a uniform brightness change shifts all gradients similarly and largely cancels out after normalization.
- Using relative rather than absolute intensity comparisons (e.g., local binary patterns, or ratio-based measures) further reduces sensitivity to global lighting change.

Conclusion

By normalizing illumination beforehand and using descriptors built from normalized gradient structure rather than raw pixel values, feature matching becomes robust to differences in lighting between the two captures.

10. PCA in Face Recognition

A face recognition system processes high-dimensional image data, resulting in slow performance. Analyze how PCA helps in reducing dimensionality while preserving important features.

Analysis of the Problem

A face image of modest resolution (e.g., 100x100 pixels) yields a 10,000-dimensional raw feature vector. Comparing or classifying faces directly in this space is computationally expensive, and much of the variation between pixels is redundant (neighboring pixels are highly correlated, and large portions of the image, such as background, contribute little discriminative information about identity).

Solution / Explanation

The classical Eigenfaces approach applies PCA to this problem directly:

- The covariance matrix of a training set of face images is computed, and its eigenvectors ('eigenfaces') represent the principal directions of variation across faces.
- Each face can then be represented compactly as a weighted combination of a small number of top eigenfaces (e.g., 50–150 components) instead of the full 10,000-pixel vector.
- Because the top components are chosen to capture the greatest variance in the training data, they preserve the dominant facial characteristics that differentiate individuals, while discarding low-variance components that mostly represent noise.

Conclusion

This dramatically reduces the dimensionality of the feature space used for recognition, cutting computation and storage requirements, while retaining the components most responsible for distinguishing one face from another — enabling much faster recognition with minimal loss of accuracy.

11. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail. Analyze the problem and explain how preprocessing enhances edge detection results.

Analysis of the Problem

Edge detectors identify boundaries based on the magnitude of the intensity gradient at each pixel. In a low-contrast image, the difference in intensity across a true object boundary is small, so the resulting gradient magnitude may fall below the detector's threshold. As a result, genuine edges are missed entirely, or only fragments of them are detected, even though the structural boundary is visually present to a human observer.

Solution / Explanation

Preprocessing enhances the intensity separation before edge detection is applied:

- Contrast stretching (linear rescaling of the intensity range to use the full dynamic range) increases the difference between adjacent regions, amplifying gradient magnitude at true boundaries.

- Histogram equalization or CLAHE redistributes intensity values to increase local contrast, particularly useful when contrast is low only in certain regions of the image.
- Gamma correction can selectively boost contrast in dark or bright regions depending on where detail is being lost.
- Unsharp masking can be applied to further emphasize edges before running the detector, and detection thresholds can be lowered slightly once contrast has been improved.

Conclusion

By increasing the intensity difference across true boundaries prior to detection, these preprocessing steps allow the edge detector's gradient threshold to correctly identify structural edges that would otherwise be missed in the original low-contrast image.

12. Corner Detection in Textured Regions

A system applies corner detection in highly textured regions, resulting in excessive keypoints. Analyze the issue and suggest methods to improve feature selection.

Analysis of the Problem

Highly textured regions (e.g., foliage, fabric patterns, gravel) contain many small local intensity variations that individually satisfy the mathematical criteria for a corner (strong gradient in two directions). This causes the detector to report a very large number of keypoints clustered densely in textured areas, many of which are redundant, unstable under small perturbations, or of low individual significance — inflating computational cost for descriptor computation and matching without proportionally improving matching quality.

Solution / Explanation

Several feature-selection strategies reduce this excess while retaining the strongest, most useful keypoints:

- Raise the corner response threshold so only points with the strongest, most confident corner response (largest Harris R value) are retained.
- Apply non-maximum suppression within a local neighborhood so that only the strongest corner in each small region is kept, rather than multiple nearby detections.
- Rank all detected corners by response strength and keep only the top-N strongest keypoints (a common practice in SIFT/ORB implementations).
- Use Adaptive Non-Maximal Suppression (ANMS) to enforce a more even spatial distribution of keypoints across the image rather than dense clustering in textured patches.
- Apply detection at a coarser scale level (larger Gaussian blur) so that only more significant, stable structures produce corner responses.

Conclusion

These techniques reduce the keypoint set to a manageable number of strong, well-distributed, stable corners, lowering computational overhead while preserving the features most useful for reliable matching.

13. Feature Matching with Noise

Feature matching between two images fails due to noise in one image. Analyze the impact of noise and explain how preprocessing improves matching accuracy.

Analysis of the Problem

Noise perturbs pixel intensities randomly, which in turn perturbs the local gradients used to detect and describe features. This has two compounding effects: keypoints may be detected at slightly different, unstable locations in the noisy image compared to the clean one, and the descriptor computed at a given keypoint (built from local gradient statistics) becomes distorted, reducing its similarity to the corresponding descriptor in the clean image. Both effects increase the rate of missed matches and incorrect matches.

Solution / Explanation

Preprocessing and robust matching strategies mitigate the effect of noise:

- Apply a denoising filter before feature detection — Gaussian or median filtering for general noise, or bilateral filtering when edge preservation is important, since it smooths flat regions while retaining sharp boundaries.
- Use descriptors that aggregate gradient information over a local neighborhood (such as SIFT's histogram-of-gradients), which naturally averages out small random noise contributions rather than relying on individual pixel values.
- During matching, apply the ratio test (comparing the best match distance to the second-best) to reject ambiguous matches likely caused by noise.
- Apply RANSAC when estimating a geometric transformation between the images, which discards outlier matches that noise may have introduced.

Conclusion

Denoising the image before detection, combined with using noise-robust descriptors and outlier-rejecting matching strategies, substantially restores matching accuracy when one image contains noise.

14. PCA and Information Loss

A system applies PCA aggressively and loses important feature details. Analyze the trade-off between dimensionality reduction and information loss, and justify optimal component selection.

Analysis of the Problem

PCA reduces dimensionality by keeping only the top-k principal components, ranked by the amount of variance they capture, and discarding the rest. If k is chosen too small, components that still carry meaningful discriminative variance are discarded along with the noise, so accuracy in downstream tasks (matching, classification, recognition) degrades even though computational cost is minimized. This illustrates the fundamental trade-off in PCA: fewer components mean greater compression and speed, but at some point further reduction removes genuine signal rather than redundancy.

Solution / Explanation

Choosing the number of components should be a deliberate, data-driven decision rather than an arbitrary aggressive cut:

- Cumulative Explained Variance: select the smallest k such that the retained components together explain a target proportion of total variance (commonly 90–99%, depending on task tolerance for information loss).
- Scree Plot / Elbow Method: plot eigenvalues in decreasing order and choose k at the point where the marginal gain in variance explained by adding another component drops sharply.
- Cross-Validation on Task Performance: rather than optimizing variance alone, evaluate the actual downstream metric (e.g., matching accuracy or recognition rate) at different values of k and pick the point where performance stabilizes.

Conclusion

This balances the two competing goals: enough components are retained to preserve the discriminative information the task needs, while components beyond that point — which contribute little but add cost — are still discarded, giving an efficient representation without sacrificing critical detail.

15. Combined Feature Detection and Matching

A system detects features correctly but fails during matching due to incorrect descriptor selection. Analyze the issue and explain how appropriate feature descriptors improve matching performance.

Analysis of the Problem

Feature detection and feature description are two separate stages: detection finds where the interesting keypoints are, while the descriptor encodes what the local neighborhood around each keypoint looks like, in a form that can be compared numerically. Even if keypoints are detected correctly and consistently in both images, matching will still fail if the descriptor chosen is not appropriate for the conditions of the task — for example, using a descriptor with no rotation invariance when the images differ in orientation, or using a computationally simple but low-distinctiveness descriptor when the scene contains many visually similar regions, leading to ambiguous matches.

Solution / Explanation

Selecting a descriptor suited to the task requirements resolves this:

- SIFT / SURF descriptors encode a normalized histogram of local gradient orientations, giving strong invariance to scale, rotation, and moderate illumination change — well suited when transformations between images are significant.
- ORB (Oriented FAST and Rotated BRIEF) produces compact binary descriptors that are fast to compute and match (via Hamming distance), and includes orientation compensation — well suited for real-time applications where speed matters more than maximal distinctiveness.
- BRIEF is fastest but lacks rotation/scale invariance, making it appropriate only when viewpoint change between images is minimal.
- The choice should match the expected variability between images (scale, rotation, illumination) and the computational budget of the application.

Conclusion

Because the descriptor — not the detector — determines how distinguishable and comparable each keypoint's local appearance is, matching a descriptor's invariance properties to the actual transformations present between the two images is essential; doing so resolves matching failures that persist even when detection itself is accurate.

Code of Conduct

I, Tamilselvan sasikumar (192524134), certify that this submission is my original work and that I have adhered to the *University of Maryland System's Code of Academic Integrity*. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tamilselvan sasikumar

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand